

Optimizing Cache in Matrix Multiplication

Assignment #2, CSC 746, Fall 2026

Chu Yi Aaron Herr
Computer Science Department
San Francisco State University
San Francisco, CA, USA
Email: cherr1@sfsu.edu

Abstract—This study evaluates and compares the performance of three matrix multiplication implementations. A naive triply nested loop matrix multiplication, cache-optimized blocked implementations that are performing local-copy optimization, and a vendor-optimized CBLAS routine to perform the matrix optimization. By using square matrices across range of problem sizes. We measure the results that demonstrate the poor cache locality and how some of the optimizations we perform improve cache utilization.

PROBLEM STATEMENT

Matrix multiplication $C = C + A * B$ is common among bottlenecks in areas such as simulation and scientific computing. These naive implementations suffer from $O(N^3)$ complexity and reduce performance from poor memory access patterns. For varied problem sizes of N , both spatial and temporal data locality degrade rapidly that can be led through bandwidth or even severe pipeline stalls.

MY APPROACH

To address these limitations, this study investigates three implementations that were provided by the code harness. The first implementation establishes a very naive implementation where we use a triply nested for-loop that performs row-col indexing. Second implementation is the blocked matrix multiplication with Copy Optimization partition matrices into sub-grids of n by n size represented as **block size**.

To prevent conflict misses and improve stride access from matrices A , B , and C are copied to contiguous buffers An , Bn , Cn using **memcpy** and subsequently writing back to global memory. Third performance that is bench-marked against the standard CBLAS reference wrapper in **dgemm-blas.cpp** leverages vectorization and architectural-specific optimizations.

EXPERIMENTATION AND RESULTS

The performance measurements shows the basic matrix multiplication implementations MFLOP/s drops as the matrices outgrow the CPU cache, mainly due to non-unit stride memory access for matrix B . The BMMCO implementation improves here by keeping those workloads smaller and cache-friendly, demonstrating how block size selection makes accessing matrices in blocks better in the context of accessing cached values. The CBLAS reference implementation outperformed the previous two implementations by leveraging vectorization such as SIMD vector instructions to improve how these perform matrix multiplication.

IMPLEMENTATION DETAILS

Basic Matrix Multiply Implementation (dgemm-basic.cpp)

This basic implementation computes $C := C + A \times B$ directly using a triply nested for-loop to iterate through each row and columns to accumulate the dot product into a local variable called **square_num**.

```
1 for (int i = 0; i < n; i++) {
2   for (int j = 0; j < n; j++) {
3     double square_sum = C[i*n + j];
4     for (int k = 0; k < n; k++) {
5       square_sum += A[i*n + k] * B[k*n + j];
6     }
7     C[i*n + j] = square_sum;
8   }
9 }
```

The biggest realization was because we are accessing matrix B 's columns using $B[k * n + j]$, this can cause poor cache utilization and heavy cache misses towards large problem sizes.

Blocked Matrix Multiply with Copy Optimization *(dgemm-blocked.cpp)*

This other implementation, we divide the matrices into smaller sub-blocks based on the given **blocked_size**. Using helper functions to handle copying and writing those blocks. Where the copy function utilizes `memcpy` to safely transfer between local matrices to the local contiguous buffers (**An**, **Bn**, **Cn**):

This next implementation, we divide the matrices operations into smaller sub-blocked based on a given **blocked_size**.

Utilizing helper functions that perform copy and write operations for each of those sub-blocks.

Where we utilize the **copy** and **write** helper functions to use `memcpy` to safely transfer between local matrices to local contiguous buffers **An**, **Bn**, **Cn**:

```
1 for (int y = 0; y < block_size; y++) {
2     std::memcpy(&dest[y * block_size],
3         &src[y * n],
4         block_size * sizeof(double));
5 }
```

```
1 for (int y = 0; y < block_size; y++) {
2     std::memcpy(&dest[y * n],
3         &src[y * block_size],
4         block_size * sizeof(double));
5 }
```

In the main **square_dgemm_blocked** routine, we first determine the total number of blocks per dimension ($total_block_size = n/block_size$) and allocate our local block buffers. The outer loops iterate over the block coordinates (i and j), computing the global memory position offset **Cpos** for result matrix **C** and copying the initial accumulation block into local buffer **Cn**.

Then, the inner loop iterates over k to fetch corresponding sub-blocks of matrices **A** and **B** into **An** and **Bn**, invoking our basic matrix multiplication kernel on the local buffers. Once all inner block products are accumulated, the final result block is written back to global memory via **write_block**:

```
1 int total_block_size = n / block_size;
2 int block_arr_size = std::pow(block_size, 2);

4 std::vector<double> An(block_arr_size);
5 std::vector<double> Bn(block_arr_size);
6 std::vector<double> Cn(block_arr_size);

8 for (int i = 0; i < total_block_size; i++) {
9     for (int j = 0; j < total_block_size; j++) {
10         // calculate this so we store it in
            register
11         int Cpos = i * n * block_size + j *
            block_size;
```

```
        copy_block(Cn.data(), &C[Cpos], n,
            block_size);

        for (int k = 0; k < total_block_size; k
            ++) {
            copy_block(An.data(), &A[(i *
                block_size * n) + (k * block_size)], n,
                block_size);
            copy_block(Bn.data(), &B[(k *
                block_size * n) + (j * block_size)], n,
                block_size);

            square_dgemm(block_size, An.data(), Bn
                .data(), Cn.data());
        }

        write_block(&C[Cpos], &Cn[0], n,
            block_size);
    }
}
```

CBLAS Reference Implementation *(dgemm-blas.cpp)*

To provide with a comparison with an industry third-party library, we compare the previous implementations with CBLAS. To provide with better timing. It was required by the assignment to discard the first initial run of the benchmark.

As to avoid skewing the timing data with the DLL loading and initialization overhead.

```
1 cblas_dgemm(CblasRowMajor, CblasNoTrans,
            CblasNoTrans, n, n, n, 1., A, n, B, n, 1.,
            C, n);
```

Comparing Basic MM and CBLAS

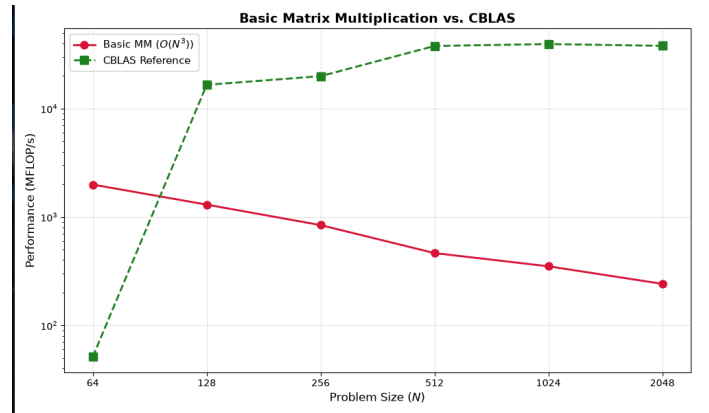


Fig. 1. Basic MM vs CBLAS Comparisons

Performance Analysis: Looking at the naive matrix multiplication implementation which uses a triply nested loop. The MFLOP/s drops as the size of the problem scales. When analyzing the scale at $N = 64$, the matrix is

able to fit into L1/L2 cache, which can mean the latency in memory is quite low and the data re-usability is quite high.

Since this implementation is $O(N^3)$ and has terrible memory access patterns – the CPU suffers from massive cache misses. Where the processor spends most of its time stalled and waiting for data to be retrieved to perform multiplication on.

1) *Comparing MM to CBLAS*: Comparing the naive implementation to the reference implementation. The CBLAS implementation immensively outperforms the initial implementation. The performance changes because there are some optimizations CBLAS is able to utilize such as vectorization or other words SIMD instructions to reduce the instructions required to perform these matrix multiplications.

Comparison of Blocked MM with Copy Optimization (BMMCO) and CBLAS

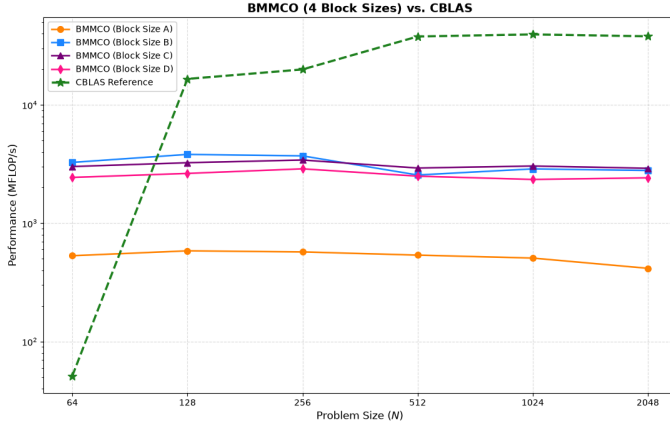


Fig. 2. Basic MM vs CBLAS Comparisons

Performance Analysis: Unlike the first implementation where we utilized a triply nested for-loop which is completely outperformed by CBLAS. This implementation variant is quite stable across various problem sizes. As soon we get passed $N = 64$, the plots in the graph start seem to look quite flat.

Comparing BMMCO to CBLAS: By copying in blocks and packing them contiguously in memory, we reduce severe cache-line misses. The CBLAS reference implementation still outperforms this blocked operation because the compiler can also outperform by auto-vectorizing code if possible. One thing that is noted from the assignment instructions, is the overhead of startup when executing CBLAS the first time. Due to DLL loading and and initialization that is required when calling the CBLAS APIs.

Overall Findings and Discussion

Comparing both the basic matrix multiplication and the blocked matrix multiplication implementations. They both share highly relative throughput when the problem sizes are $N = 64$ where the data is able to fit into the CPU's L1 cache. However, the basic matrix multiplication suffers immensively when the problem sizes scale due to cache thrashing and memory latency. The BMMCO implementation breaks down the matrices into sub-blocks and packs them to contiguous memory. This copy optimization allows for the performance to stabilize as shown in the chart, where each block sizes are shown to be quite stable and flat.

From analyzing the datasets performance is driven from hardware characteristics when workloads exceed L2/L3 cache sizes. Which can cause high latency on the AMD EPYC processors. Whereas BMMCO mitigates these cache misses through tiling. Despite the implementation breaking down to sub-blocks these two implementations are still outperformed by the CBLAS reference implementation. By making use of SIMD instructions.